
1:21 PM
81%

Print Binary String (No consecutive 1's)

→ Size
N = 3

101
001
000
010
100

N=0 → ""
N=1 → "1" "0"
N=2 → "10" "00" "01"

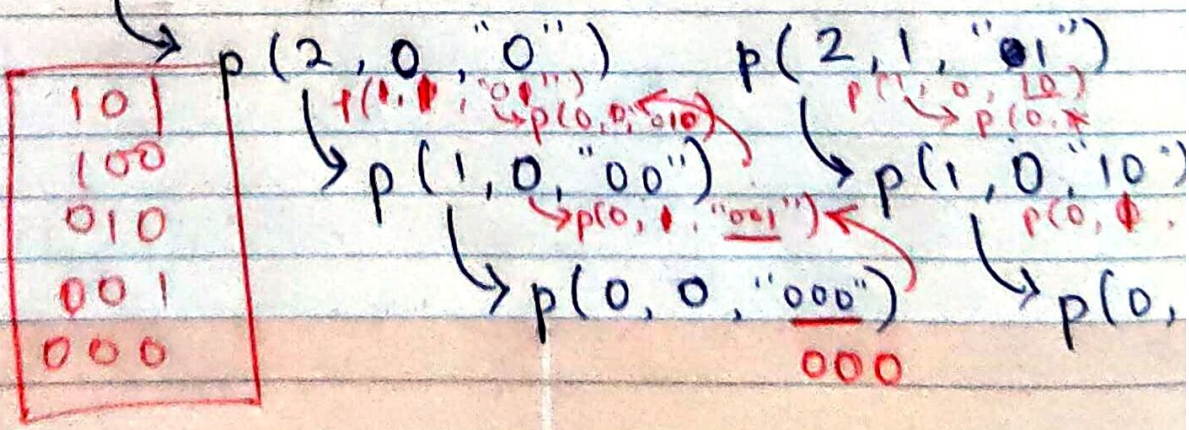
1) Base Case –

(N==0) sop(str) return

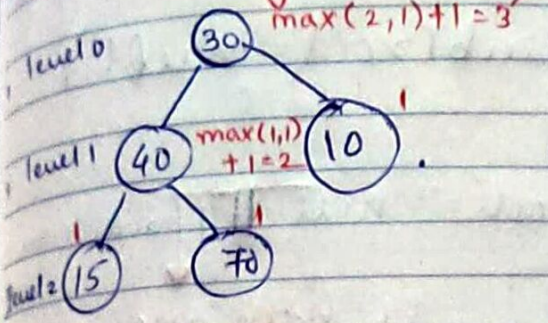
2) if (lastPlace == 1)
print(n-1, 0, str+"0");

else {
3) print(n-1, 0, str+"0")
print(n-1, 1, str+"1").
4) }

5) print(3, 0, "")



Binary Tree -



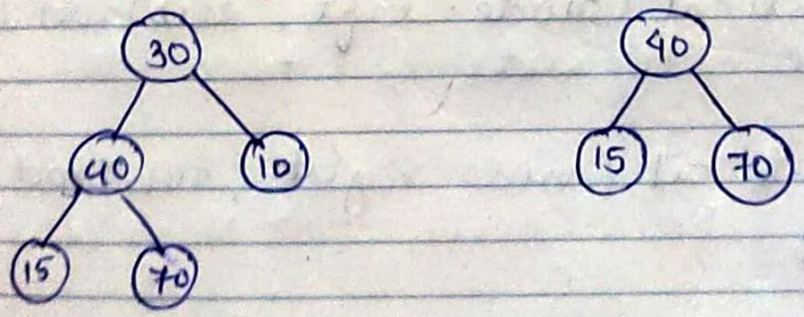
pre - 30, 40, 15, 70, 10
post - 15, 70, 40, 10, 30
in - 15, 40, 70, 30, 10
level - 30, 40, 10, 15, 70

height = max(lefth, rigthh) + 1

Diameter is the longest distance/path b/w any two nodes in tree

- For above example, diameter = 4.

Subtree of another tree -



Sunday 16

① If root == null → return false

② root.data = subRoot.data.

Yes → check if they are identical or no.

③ No → Go for left subtree & do same
Then go for right subtree & do same

④ Return leftAns || rightAns, atleast any 1 tak will be subtree, else false.

To achieve great things we have first to believe it

To check not identical →

```
if(root == null && subTroot == null)
    ↳ true

else if( node==null || subTroot == null || node.data!=subTroot.data)
    ↳ false
```

if both are null then we return 'True', if not means any one of them is !null so it can be subtree, so return false & it goes to caller so that next left & right subtrees can be checked

if not →

```
if (!isIdentical(node.left, subTroot.left)
        false

if (!isIdentical(node.right, subTroot.right)
        false

return true;
```

JUNE
19 2019
Wednesday
Wk 25 • Day (170-195)

7am

Level Oxeder -

- As its BFS, we need to use a Queue.

1) Root == null
   → Yes → return.
2) → No → Create a queue

3) Add root node & a null (∵ we need next line after each level)
   q.add(root)
   q.add(null).

4) We'll check, it queue is not empty →

i) Remove 1st ele/node & store it as cur.

ii) Check if cur == null —Yes→ Next line.

iii) Now, we have removed it, then if queue is empty
   Yes → break (∵ queue is empty)
   No → add(null) (∵ there are elements)

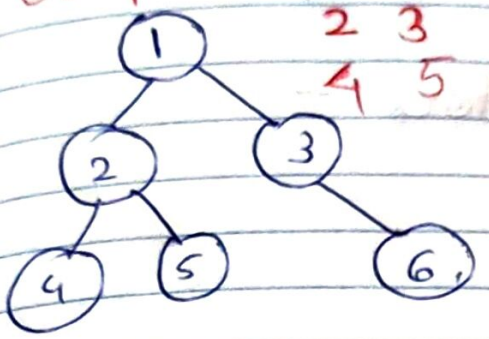
@

No → Sop(cur.data + " ");
(cur.left != null) → q.add(cur.left)
(cur.right != null) → q.add(cur.right)

Plan to succeed. If you fail to plan, then you plan to fail

End.

Output → 1 2 3 4 5 6



| 1 | N |

| N |

↳ cur = 1

| N | 2 | 3 |

| 2 | 3 | N |

↳ cur = null ⇒ sop

| 3 | N | 4 | 5 |

↳ cur = 2

| N | 4 | 5 | 6 |

↳ cur = 3

| 4 | 5 | 6 | N |

↳ cur = N ⇒ sop

| 5 | 6 | N |

↳ cur = 4

| 6 | N |

↳ cur = 5

| N |

↳ cur = 6

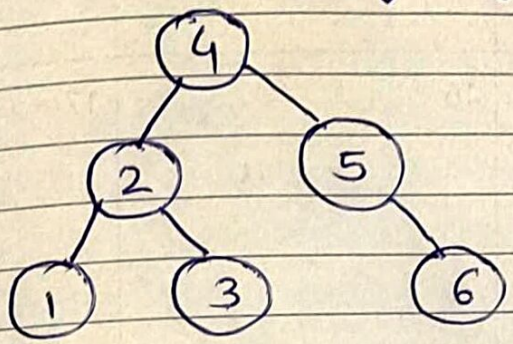
← queue is empty break if we're done :

↳ cur = N ⇒ sop

Binary Search Tree -

- Left subtree nodes < Root
- Right subtree nodes > Root.
- BST has at max 2 childrens only.

- Inorder always gives sorted.



Inorder - 1, 2, 3, 4, 5, 6

- Balanced BST → Searching ⇒ O(log n) ← average case
- Skewed BST → Searching ⇒ O(n) ← worst case
- So, BST searching, TC = O(H) *

Sunday 30

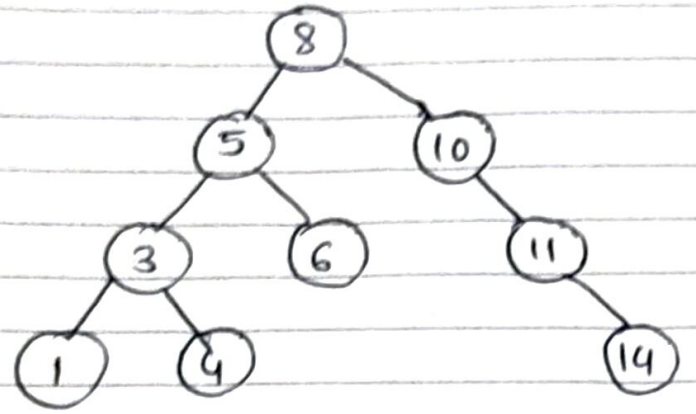
Deleting a node in BST -

Cases -

1) No child (leaf node)
2) One child
3) Two child.

JULY
01 2019
Monday

Wk-27 • Day (182-183)



1) No child → 1, 4, 14
2) One child → 10, 11
3) Two child → 8, 3,

Case 1 →
- Delete node
- Return null

Case 2 →
- Delete node
- Replace with child node

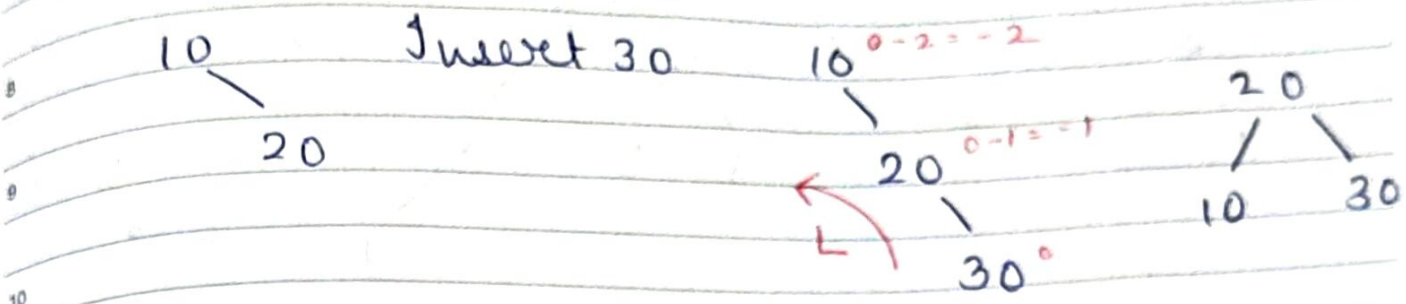
Case 3 →
- Replace value with inorder successor
- Delete the node for inorder successor

* inordere successore always has 0 or 1 chil

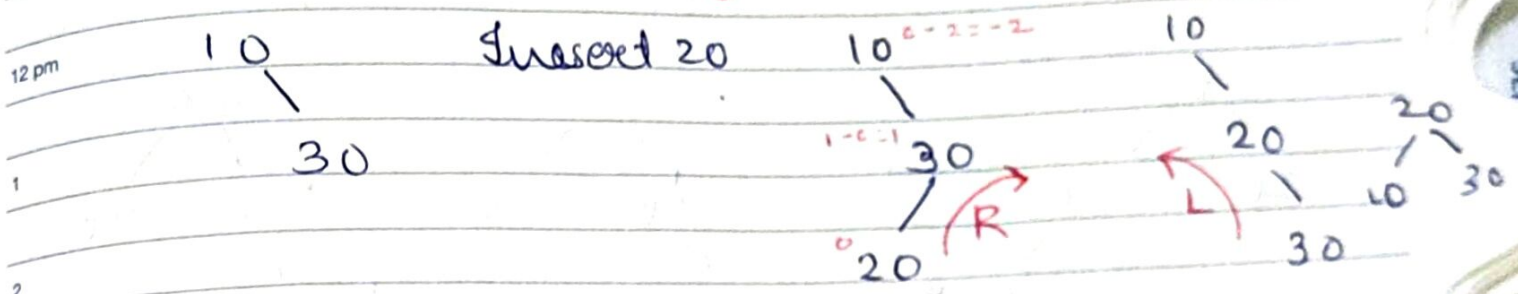
Inorder Successor → In inorder, the node next to the node we wanna delete
Ex → 1, 3, 4, 5, 6, 8, 10, 11, 14.
if we wanna delete 5 ⇒ IS = 6

- Leftmost Node in right Subtree.
(Min value in right Subtree).

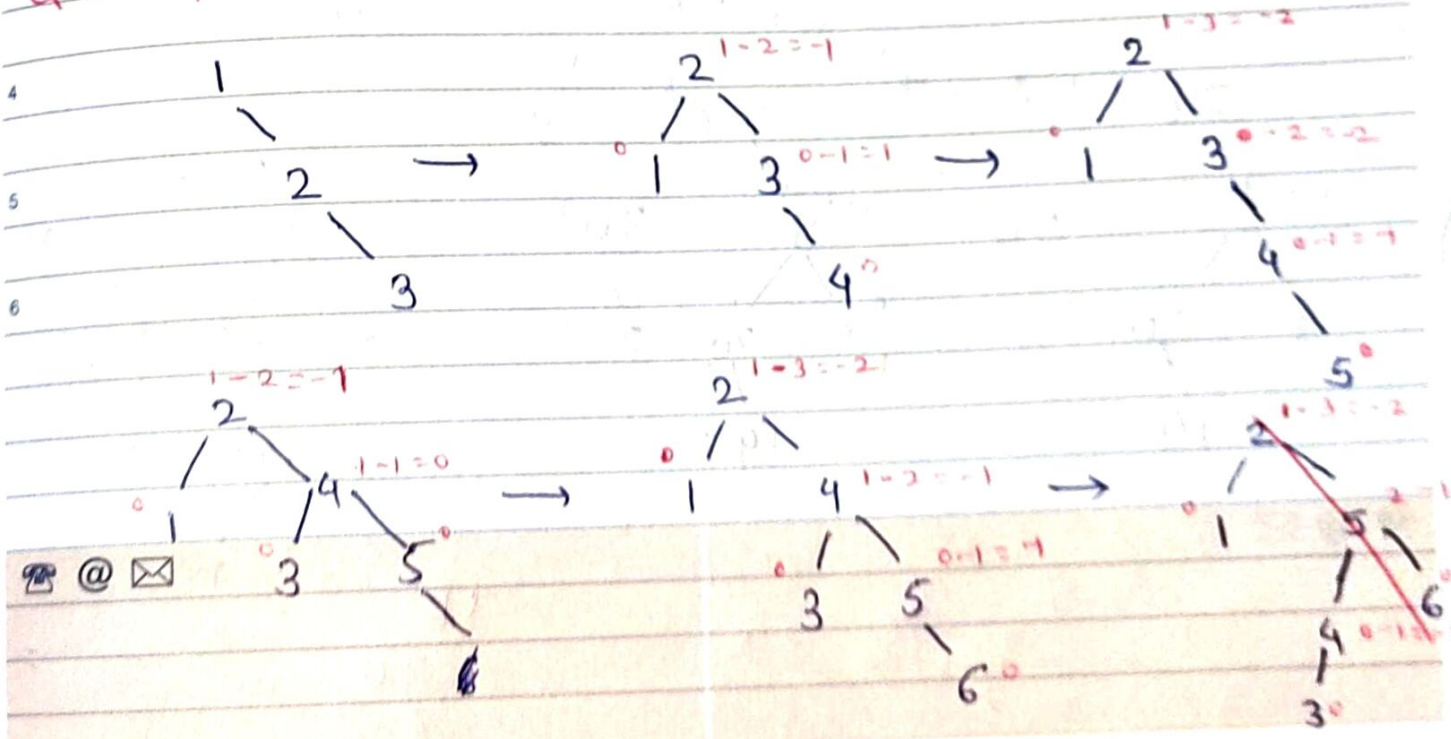
3) RR case → Left Rotation



4) RRL case → Right Rotation, Left Rotation



Q. 1, 2, 3, 4, 5, 6, 7



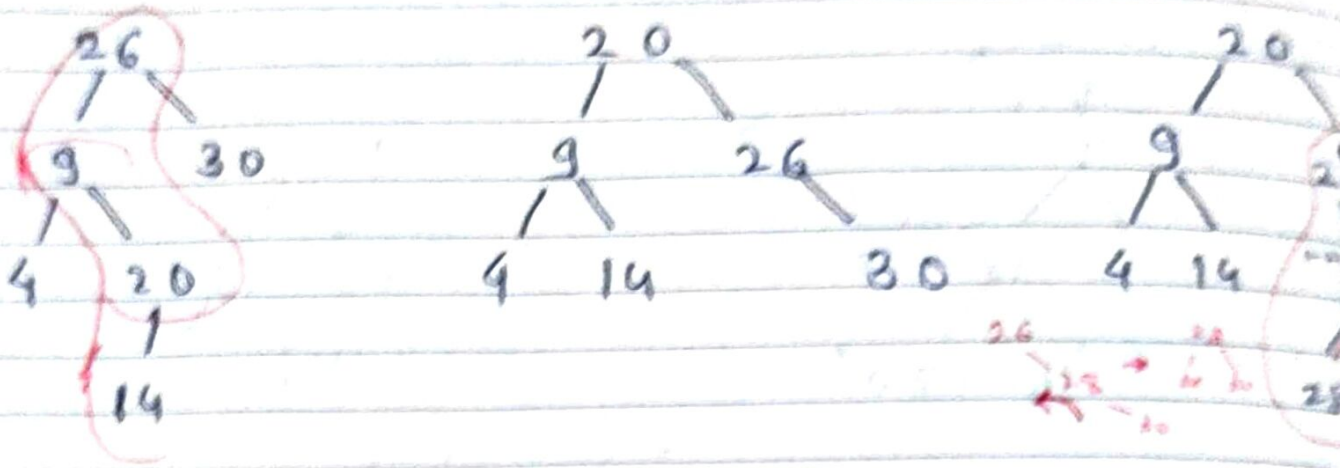
A positive approach can recharge the low esteem of a team, keep the team going forward

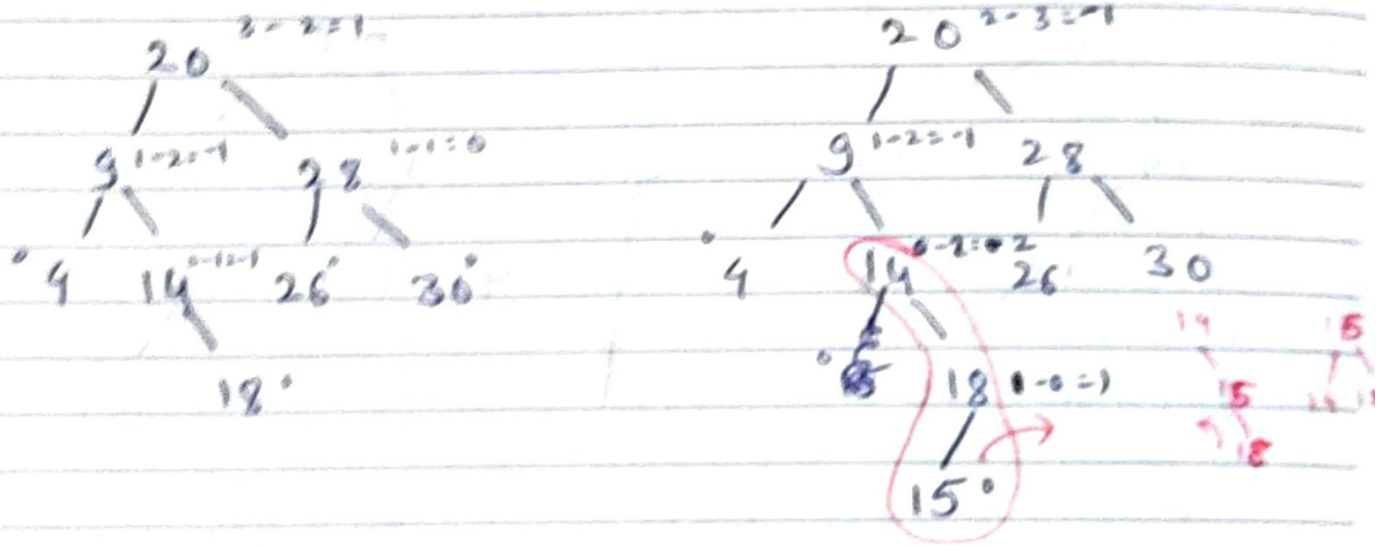
JULY

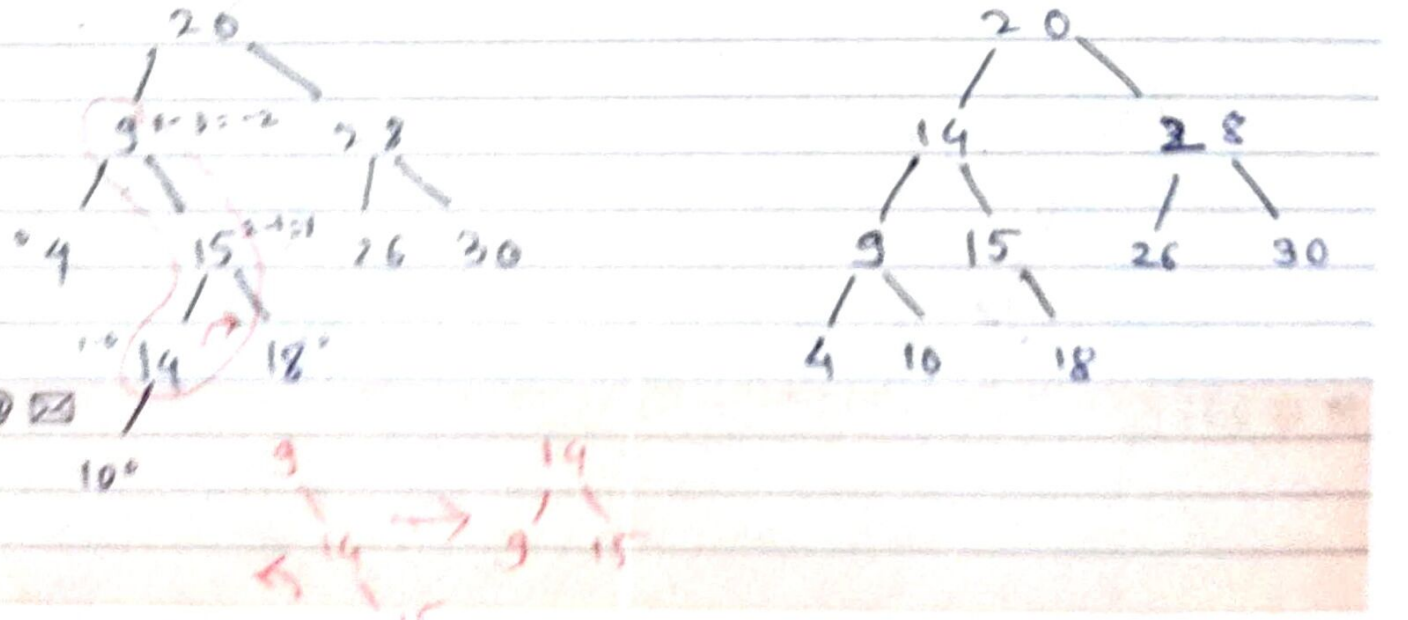
12 2019 Friday



30, 26, 30, 9, 4, 14, 28, 18, 15, 10, 2, 3, 4



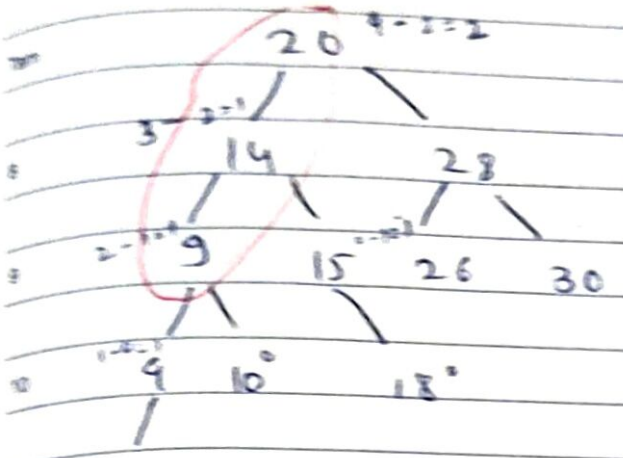


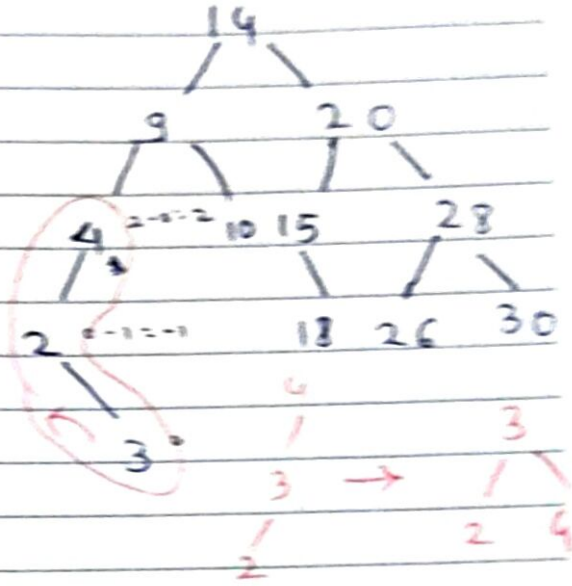


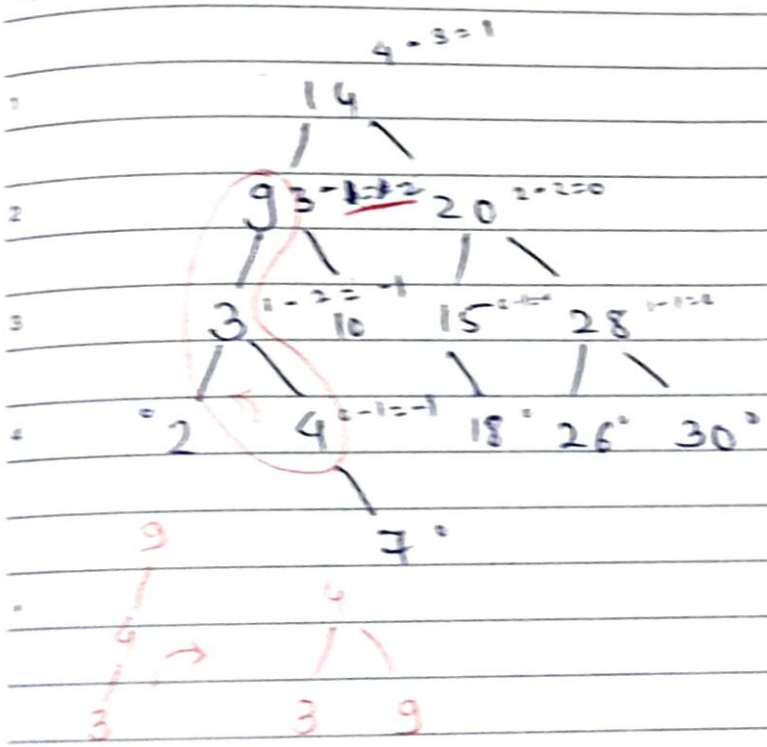
August 2019 | September 2019

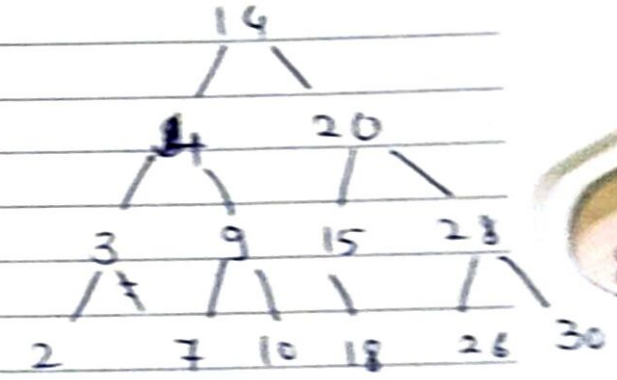
JULY

2019 Saturday 13









Sunday 14

Don't fear the obstacles, they make you nervous but can never stop you from winning

JULY

10 2019 Wednesday

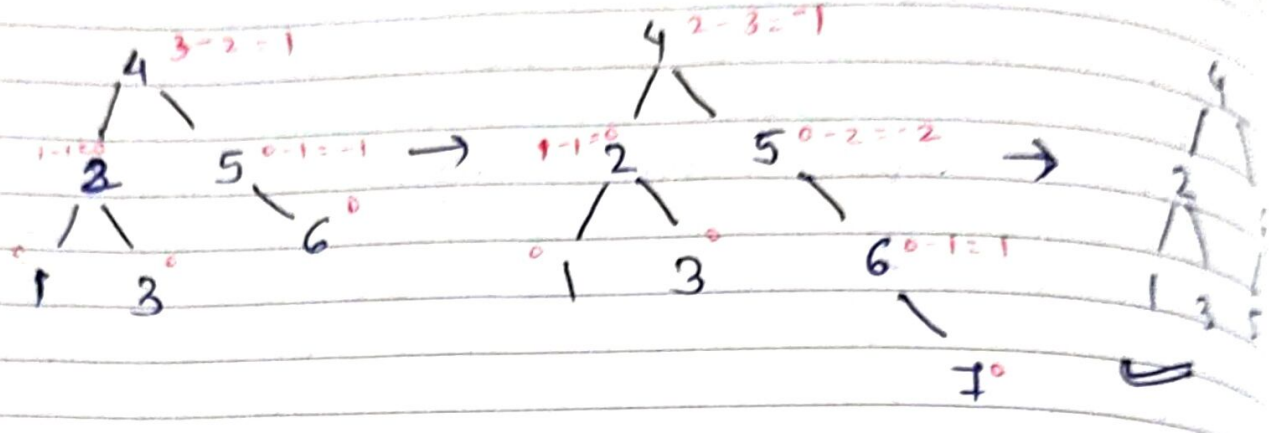
Wk-28 • Day (191-174)

June 2019

Wk	M	T	W	T	F	S	S
22						1	2
23	3	4	5	6	7	8	9
24	10	11	12	13	14	15	16
25	17	18	19	20	21	22	23
26	24	25	26	27	28	29	30

Wk	M	T	W	T	F	S	S
27	1	2	3	4	5	6	7
28	8	9	10	11	12	13	14
29	15	16	17	18	19	20	21
30	22	23	24	25	26	27	28
31	29	30					

7am



8

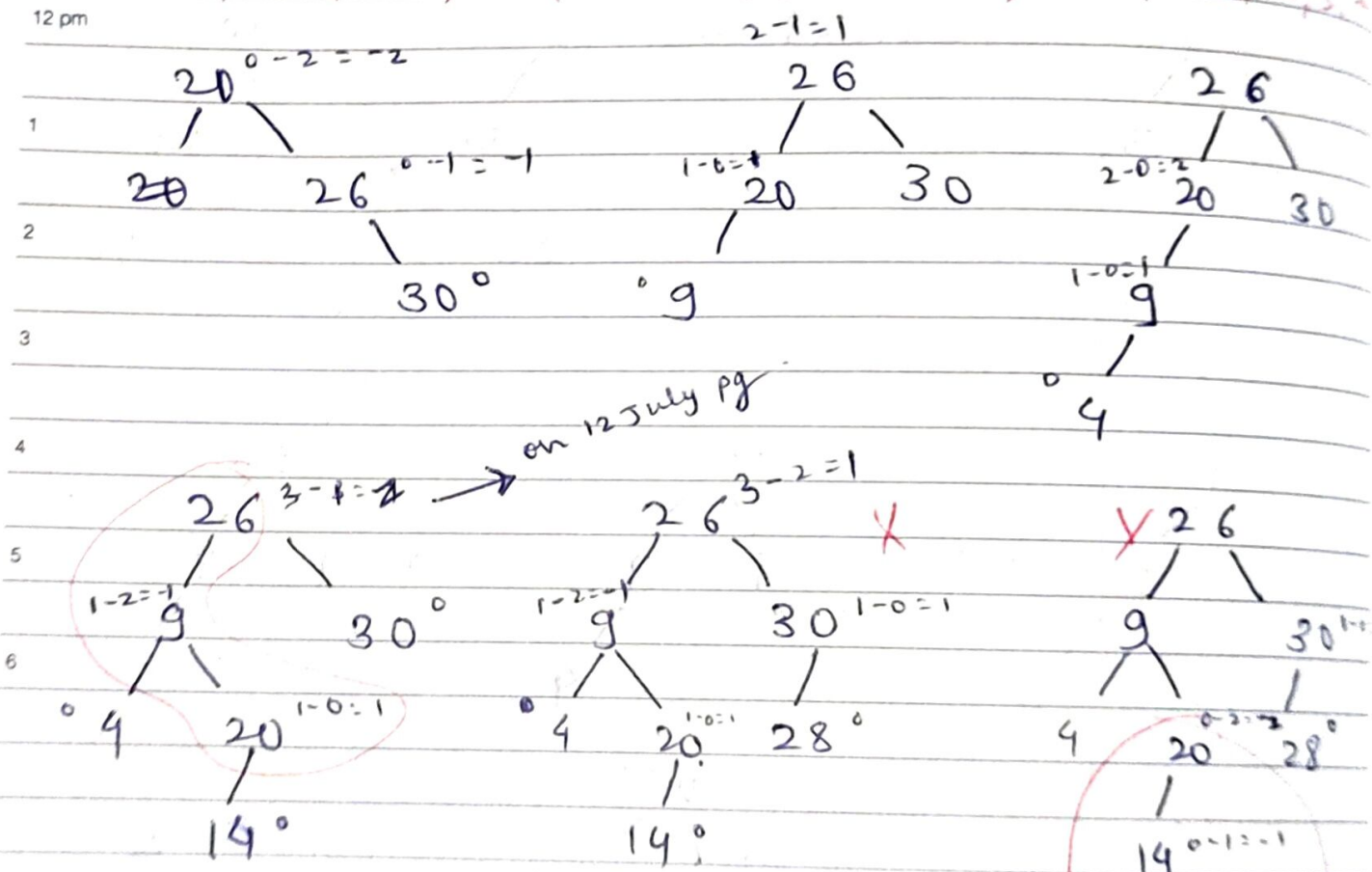
9

10

11

Q. 20, 26, 30, 9, 4, 14, 28, 18, 15, 10, 23

12 pm



1

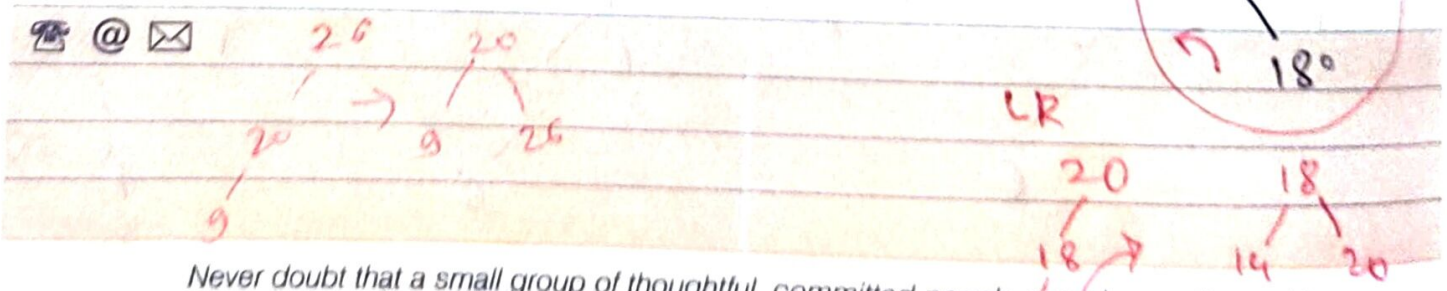
2

3

4

5

6



Never doubt that a small group of thoughtful, committed people can change the world

JULY

08 2019 Monday

WK 28 • Day (189-176)



AVL TREES -

- Balance factor = height of left subtree - height of right subtree
(-1, 0, 1).

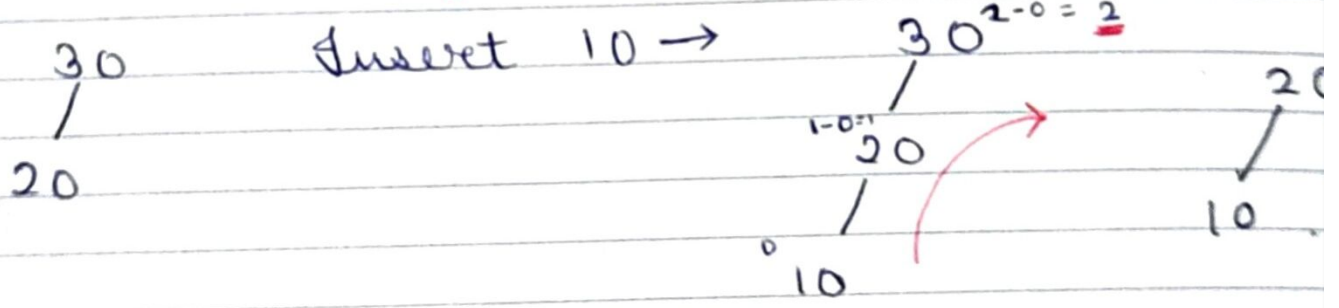
- self Balancing trees.

- why they are introduced →

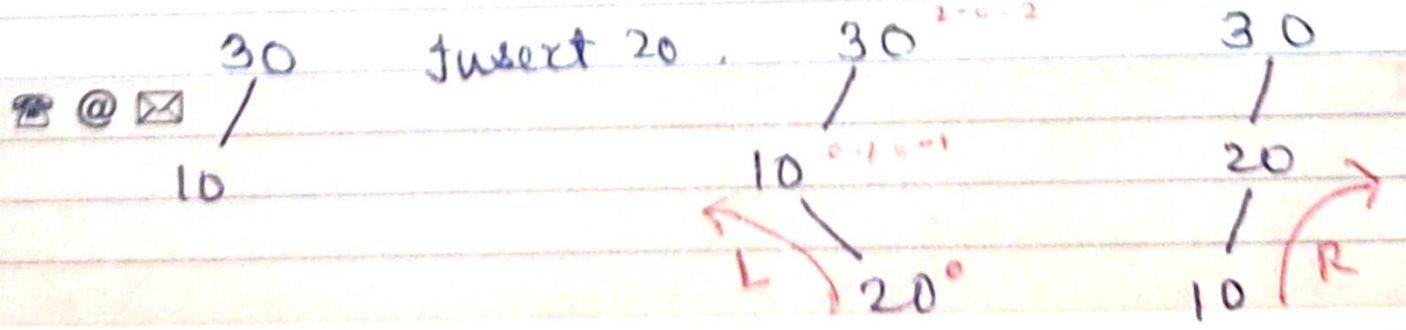
As TC of trees depends on height, for a normal TC = O(log n) but for skewed BST, it becomes TC = O(n²).

Cases -

1) LL case : → Right Rotation



2) LR case → Left Rotation, Right Rotat



The achievements of an organiztion are the results of the combined efforts of each individual